



ANNO ACCADEMICO 2025/2026

CORSO DI TECNICHE DI PROTEZIONE DEL SOFTWARE
MEMORY SAFETY IN RUST



Marco Agnoli

OBIETTIVO

Il lavoro intende mostrare come i meccanismi interni di Rust — come ownership, borrowing, scope e lifetime — permettano di garantire memory safety già in fase di compilazione, riducendo drasticamente errori come buffer overflow, use-after-free, double free e dangling pointer e impedendo gli attacchi visti a lezione. Il tutto, con performance da linguaggio C.

Abstract geometric lines in the top left corner, consisting of several overlapping, irregular polygons and lines in a light beige color, creating a layered, architectural feel.

TEMPORAL SAFETY IN RUST



OWNERSHIP

In Rust ogni valore ha un proprietario attivo, detto owner. Il compilatore è responsabile della distruzione del valore quando l'owner esce dal suo scope.

OWNERSHIP

C/C++

```
int main() {  
    char *s = malloc(5); // alloca 5 byte sull'heap per contenere la stringa "ciao"  
    strcpy(s, "ciao");    // copia la stringa "ciao" nella memoria puntata da s  
    char *t = s;          // copia il puntatore: t ora punta alla stessa memoria di s  
    printf("%s\n", t);    // stampa la stringa tramite il puntatore t  
    printf("%s\n", s);    // stampa la stessa stringa tramite il puntatore s  
}
```

Rust

```
fn main() {  
    let s = String::from("ciao"); // alloca una String sull'heap e assegna ownership a s  
    let t = s;                    // trasferisce l'ownership della stringa da s a t  
    println!("{}", t);           // uso corretto: t è ora il proprietario della stringa  
    println!("{}", s);           // errore di compilazione: s non è più valida dopo il move  
}
```



BORROWING

BORROWING

Permettere l'uso temporaneo di un valore senza trasferirne l'ownership

MUTABLE

Un riferimento mutabile non permette solo di accedere al dato, ma di modificarlo, e per questo richiede esclusività. Possono esistere tanti riferimenti unmutable ma uno solo mutable (*aliasing XOR mutability*)

ESCLUSIVITA'

Sia la «mutability» che «drop» richiedono esclusività



I RIFERIMENTI NON DEVONO SOPRAVVIVERE AI VALORI REFERENZIATI (LIFETIMES)

```
fn main() {  
    let r; // dichiara una variabile che conterrà un riferimento  
    {  
        let x = 42; // x vive solo all'interno di questo blocco  
        r = &x; // prova ad assegnare a r un riferimento a x  
    } // qui x esce dallo scope e viene distrutto  
    println!("{}", r); // errore di compilazione: r vivrebbe più a lungo di x  
}
```

Abstract geometric lines in the top left corner, consisting of several overlapping, irregular polygons and lines in a light beige color.

PROTEZIONE DA
ATTACCHI

USE AFTER FREE (HEAP E STACK)

C/C++

```
int main() {
    int *p = malloc(sizeof(int)); // alloca un intero sull'heap e assegna il puntatore a p
    *p = 42;                      // scrive il valore 42 nella memoria allocata
    int *q = p;                   // copia il puntatore: q punta alla stessa memoria di p
    free(p);                     // libera la memoria heap
    printf("%d\n", *q);           // use-after-free: q dereferenzia memoria già liberata
}
```

```
int* dangling() { // funzione che restituisce un puntatore a int
    int x = 5;    // variabile locale allocata nello stack
    return &x;    // restituisce l'indirizzo di x
}
```

Rust

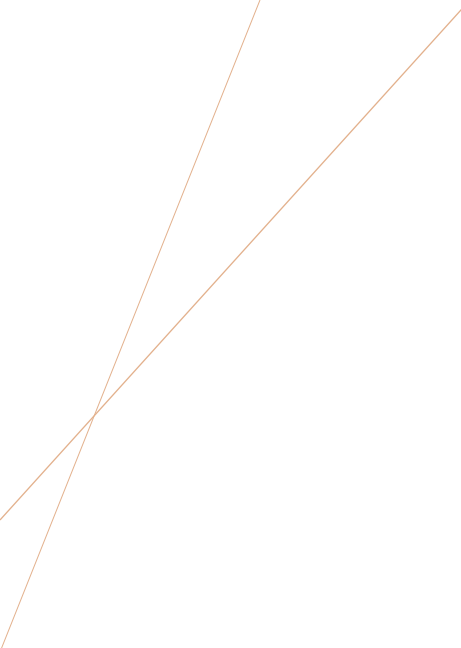
```
fn main() {
    let p = Box::new(42); // alloca un intero nello heap
    let q = &p;            // q prende in prestito p
    drop(p);              // ERRORE: non posso distruggere p mentre è preso in prestito
    println!("{}", q);    // q userebbe un dato che sarebbe stato liberato
}
```

```
fn dangling() -> &i32 { // funzione che restituisce un riferimento
    let x = 5;          // variabile locale nello scope della funzione
    &x                  // ERRORE: riferimento a x uscirebbe dallo scope
}
```

I riferimenti non devono sopravvivere ai valori referenziati
+
Aliasing XOR mutability

Abstract geometric lines in the top left corner, consisting of several overlapping, irregular polygons and lines in a light beige color.

SPATIAL SAFETY IN RUST



SPATIAL SAFETY

NESSUNA
DEFERENZIAZIONE E
ARITMETICA DEI
PUNTATORI

ASTRAZIONI E WIDE
POINTERS

BOUNDS CHECKER

PARTE SPERIMENTALE

UTILIZZO DI JULIET C/C++ 1.3

CWE SCELTE MAPPANO QUELLE VISTE A LEZIONE

#	Vulnerabilità	CWE	File Juliet	Cosa succede in C	Compila in Rust?	Panic in Rust	Cosa succede in Rust
1	Double Free	CWE-415	CWE415_Double_Free__malloc_free_char_01.c	La stessa area heap viene liberata due volte; possibile corruzione dell'heap o crash	No	No	In safe Rust il secondo drop viola «Il trasferimento di ownership non deve lasciare accessi non validi» : errore a compile time
2	Use-After-Free	CWE-416	CWE416_Use_After_Free__malloc_free_char_01.c	Il programma usa un puntatore dopo free; comportamento indefinito, leak o code execution	No	No	L'istruzione dopo il drop viola «I riferimenti non devono sopravvivere ai valori referenziati» : errore a compile time
3	Dangling Pointer / Return of Stack Variable Address	CWE-562	CWE562_Return_of_Stack_Variable_Address_01.c	La funzione restituisce l'indirizzo di una variabile locale non più valida	No	No	Il tentativo viola «Il trasferimento di ownership non deve lasciare accessi non validi» : errore a compile time
4	Stack Buffer Overflow	CWE-121	CWE121_Stack_Based_Buffer_Overflow__char_type_overflow_memcpy_01.c	Scrittura oltre i limiti del buffer sullo stack	Sì	Sì	In Rust safe non c'è corruzione silenziosa: il metodo copy_from_slice viene bloccato a run time dal bounds checker e il programma va in panic
5	Heap Buffer Overflow	CWE-122	CWE122_Heap_Based_Buffer_Overflow__char_type_overflow_memcpy_01.c	Scrittura oltre i limiti del buffer sullo heap	Sì	Sì	

PRESTAZIONI: ZERO-COST ABSTRACTIONS

Rust: no garbage collector

Sicurezza garantita (in gran parte) a compile time — nessun runtime overhead strutturale.

Evidenze benchmark

- Is Rust C++-fast (2022): performance simile a C++, con casi in cui Rust è più veloce
- NAS Parallel Benchmarks (2025): +1,23% rispetto a Fortran, ma 5,59% più veloce di C++
- Traduzione C→Rust (NDSS 2025): Le versioni safe mantengono prestazioni comparabili al C

Mini benchmark

Il bounds checking ha costi irrisori

```
marco@MarcoPC UCRT64 /c/Users/marco/OneDrive - Unive
$ ./bounds_benchmark.exe
safe      media:  39.64 ms  stddev:  0.96 ms
unsafe    media:  42.84 ms  stddev:  2.47 ms
iter      media:  42.88 ms  stddev:  3.58 ms
```



LIMITI DI RUST

UNSAFE RUST

```
fn main() {  
    let uart_status = 0x6000_0000 as *const u32; // registro ESP32 (indirizzo fittizio)  
  
    // let value = *uart_status; // ERRORE: dereferenziazione non consentita  
  
    unsafe { // accesso hardware  
        let value = *uart_status; // lettura registro  
        println!("Valore: {}", value);  
    }  
}
```

UNSAFE

Definisce un blocco di istruzioni all'interno del quale il compilatore non controlla più l'utilizzo della memoria in termini di sicurezza

PERCHE'?

1. Codice kernel, driver di basso livello
2. Implementazioni di astrazioni low-level efficienti (es. `Box<T>`)
3. Interoperabilità con codice C (Foreign Function Interface, FFI)

CONSEGUENZE

Il Trusted Computing Base (TCB) di Rust è l'unsafe code.

RIASSUMENDO: CONCLUSIONI

Regolarità sistemica

Forte correlazione strutturale tra natura della vulnerabilità e fase nella quale Rust la riconosce
– in un insieme rappresentativo ma limitato

Performance in linea

Benchmark dimostrano prestazioni in linea con C

Limiti

Codice unsafe definisce la Trusted Computing Base del linguaggio

Meccanismi Rc, Arc, RefCell possono causare memory leak

Potenzialità future

Microsoft e Google sono due «sponsor» con enorme influenza e stanno dando grande visibilità e popolarità al linguaggio. Ottime prospettive di utilizzo.

A series of thin, light brown lines forming an abstract, overlapping geometric pattern on the left side of the slide. The lines intersect to create various polygonal shapes, some of which are filled with a very light brown color.

GRAZIE